TelaDois — Melhorias planejadas

Gerado em 22/10/2026 (PC do trabalho) · branch `etapa-5b-monitor-virtual` · arquivo `melhorias/melhorias-planejadas.pdf`

> **Como retomar:** abra o Claude no projeto e diga *"vamos continuar as melhorias — leia `melhorias/melhorias-planejadas.html`"*. Nada destas três melhorias foi executado ainda: o próximo passo é **escolher a ordem** (seção 1) e responder as perguntas em aberto da melhoria escolhida.

1. Visão geral e ordem recomendada

#	Melhoria	Estimativa	Mexe em
A	Relógio igual ao StandBy original do iPhone	3–5 h (1 sessão)	só o app do iPhone
—	Terminar a Etapa 5d (atraso e engasgos) — já em andamento	≈ 1 dia	app do PC
B	Kit de instalação para levar o TelaDois a outro PC	≈ meio dia (1 sessão)	scripts + publicação do app do PC
C	Versão Android com as mesmas funções	7–10 dias de trabalho	app Android novo + app do PC

Ordem recomendada: Relógio → terminar a 5d → Kit → Android.

- O relógio é rápido, visível e independente (só iPhone).
- O kit e o Android dependem do **app do PC na versão final**, que a 5d ainda vai mudar.
- O Android é o maior: começa com o app do PC estável.

Os tempos incluem meu trabalho + esperas reais: build do iPhone na nuvem (~15 min por rodada), instalar no celular e você testar.

2. Pendências que já estavam abertas

- **Etapa 5d** (ordem): bancada do "parado" com o PC parado (~8 min, sair do TelaDois antes) → aplicar a correção que tirar os ~280 ms do "mouse parado que começa a mexer" + 5 ajustes menores da revisão → revisão final do branch → Task 6 (foto do cronômetro PC + iPhone, uso real, docs, 3 workflows verdes).
- **Task 9 da Etapa 5c**: teste formal do monitor virtual (some do Windows quando desligado, reiniciar e o "3" não volta, Win+Shift+[/], "Sair" não deixa FFmpeg aberto).
- **PC de casa (AMD)**: rodar `tools\vdd\instalar-vdd.ps1` como administrador e testar lá.
- **Juntar ao `main`**: só com `ios-test`, `ios-build` e `windows-test` verdes e com a sua aprovação.
- Depois: 5e (atrasar só o som do programa no iPhone, se ainda precisar), Etapa 3 (Spotify), Etapa 6 (acabamento, iniciar com o Windows, revisão de segurança).

3. Melhoria A — Relógio igual ao StandBy original 3–5 h



Hoje (TelaDois)



StandBy original do iPhone (referência)

Diferenças observadas

	TelaDois hoje	StandBy original
Cor dos dígitos	azul `#7FA9F2`	azul-claro / ciano, mais luminoso
Formato dos dígitos	condensados, cantos retos, esticados 1,5×	cantos arredondados, traço mais grosso
Tamanho	menor	ocupa quase toda a altura da tela
Data e temperatura	azuis	brancas
Canto superior direito	anel verde da bateria	vazio
Canto inferior direito	"PC conectado"	⏰ próximo alarme (12:56)

> **Limitação do iOS:** nenhum app pode ler os alarmes do app Relógio (a Apple não oferece essa função). O "⏰ 12:56" **não pode ser copiado**; no lugar dele dá para mostrar outra informação (bateria em texto, "PC conectado" ou nada).

Como seria feito

1. Ajustar cor, fonte (arredondada, do próprio sistema) e tamanho em `Theme.swift` / `ClockPageView.swift` e atualizar `docs/design/tokens.md`.
2. Reorganizar a coluna da direita (data/temperatura brancas; decidir bateria e o lugar do alarme).
3. Build e prints no CI → comparar com a foto do original → instalar no iPhone para você aprovar.

Perguntas em aberto

- ☐ Copiar tudo do original ou só a cor/fonte/tamanho dos dígitos?
- ☐ Manter o anel da bateria? (Se sim, onde.)
- ☐ O que mostrar no lugar do alarme: bateria em texto, "PC conectado" ou nada?
- ☐ "PC conectado" continua aparecendo? (Discreto, só quando conectado?)

4. Melhoria B — Kit de instalação para outro PC ≈ meio dia

Onde está o driver hoje

Não existe um arquivo do driver guardado no PC. O instalador `tools\vdd\instalar-vdd.ps1` baixa, a cada instalação, o pacote oficial `VirtualDisplayDriver-x86.Driver.Only.zip` (versão 25.7.23, 132 KB, assinado pela SignPath Foundation) do GitHub e confere o SHA-256. Depois de instalado, o driver fica guardado no próprio Windows. As configurações ficam em `C:\VirtualDisplayDriver\vdd_settings.xml`.

O que um PC novo precisa hoje (5 peças separadas)

Peça	Como é hoje
Driver do monitor virtual + tarefas `\TelaDois\`	script do repositório, como administrador, precisa de internet
`TelaDois.exe`	precisa do .NET 10 instalado
FFmpeg	`winget install Gyan.FFmpeg`, à parte
Driver USB da Apple (Apple Mobile Device Support)	iTunes/iCloud versão web, à parte
Atalho na Área de Trabalho	à mão

Proposta recomendada: "Kit TelaDois" (pasta zipada)

- `Instalar TelaDois.cmd`: pede administrador uma vez e faz tudo.
- Driver **dentro do kit** (sem download; SHA-256 e assinatura conferidos).
- TelaDois em **arquivo único que não precisa do .NET**, copiado para `%LOCALAPPDATA%\Programs\TelaDois`.
- Confere FFmpeg e driver da Apple; instala pelo `winget` o que faltar.
- Cria o atalho na Área de Trabalho e, se você quiser, abre junto com o Windows.
- `Desinstalar TelaDois.cmd` que desfaz tudo.
- O GitHub monta o zip sozinho a cada versão: é só baixar e levar no pendrive ou no Google Drive.

Alternativa: instalador `.exe` com assistente (Inno Setup). Mais bonito, mas +1 dia e o Windows mostra "editor desconhecido" (sem assinatura).

> **Atenção no PC do trabalho:** o Controle de Aplicativos do Windows pode bloquear scripts e programas sem assinatura (já bloqueou o `graphify.exe`). O kit precisa ser testado nos dois PCs.

Perguntas em aberto

- ☐ Zip + `.cmd` (recomendado) ou instalador `.exe`?
- ☐ Abrir o TelaDois junto com o Windows já no kit?
- ☐ FFmpeg dentro do kit (~150 MB, funciona sem internet) ou instalado pelo `winget`?

5. Melhoria C — Versão Android 7–10 dias

O que já existe

Projeto irmão em `C:\Pedro\Arquivos Pedro\Aplicativo Android` (repositório `Pedro3720/teladois-android`), **pausado na Etapa 0**: app "Olá" em Kotlin + Jetpack Compose, CI verde no GitHub, especificação própria e guia de instalação por `adb`. A Task 7 (instalar no celular) ficou esperando um Android com Depuração USB ligada.

O que se reaproveita e o que é novo

Reaproveita	É novo
Quase todo o app do PC: captura, compressão, monitor virtual, atalhos, tela preta, medidores	App do celular inteiro em Kotlin
Protocolo PC ↔ celular (mesmas mensagens e bytes)	Cabo pelo **ADB** em vez do serviço da Apple (exige "Depuração USB" ligada uma vez)
Design (cores, layout, comportamento do Descanso)	Decodificar o vídeo H.264 no Android (`MediaCodec`)

Etapas e estimativas

Etapa	Estimativa
Fechar a Etapa 0 + instalar no seu Android	2–3 h
Modo Descanso (relógio, clima, bateria, tela preta após 60 s)	1–2 dias
Túnel pelo cabo (ADB) no app do PC — que passa a atender iPhone **e** Android	1–2 dias
Vídeo (H.264 no Android) + monitor estendido + tela preta	2–3 dias
Ajuste de atraso no aparelho + revisão + docs	1–2 dias

Recomendação: um único app no PC que reconhece os dois celulares (a especificação do Android previa um app de PC separado, o que duplicaria o trabalho das Etapas 5a–5d). Diferente do iPhone, o Android compila e instala direto deste PC (sem nuvem e sem renovar a cada 7 dias).

> **Requisito:** um **celular Android de verdade** para testar vídeo e atraso (Android 10 ou mais novo).

Perguntas em aberto

- ☐ Qual celular Android será usado nos testes (modelo / versão do Android)?
- ☐ Um único app no PC para os dois celulares (recomendado) ou separado?
- ☐ Mantém os extras da especificação Android (contadores de notificações, música de qualquer player) ou primeiro só o que o iPhone já tem?

6. Registro de 22/10/2026 — a Segunda Tela parou de conectar

- 08:40 — o app do iPhone saiu da tela (provavelmente para a foto do StandBy original); às 08:41 reconectou. Às 08:44:39 o Windows informou que o **iPhone saiu do cabo** (o número da conexão mudou de 9 para 10, sinal de que o USB caiu e voltou); o TelaDois desligou o monitor virtual e ficou esperando, como previsto.
- 08:47 — o TelaDois foi fechado (“Sair”, para rodar o instalador do driver de novo) e não foi reaberto.
- 08:54 — reaberto pelo Windows: o iPhone conectou na hora e o monitor estendido voltou.

Lição: depois de rodar o instalador do driver, reabrir o TelaDois pelo atalho da Área de Trabalho. Com o “iniciar com o Windows” (kit ou Etapa 6) isso deixa de ser preciso.